

Taxi Revenue Efficiency & Vehicle Positioning Analytics

Group 7 · Binny Bajaj · Michael Miranda · Vincent Hayes Bursey

MGMT 59900 — Big Data Analytics in the Cloud · Summer 2026



The Business Problem

Taxi operators measure success with gross revenue and total trip counts — metrics that hide the operational inefficiency underneath.



Idle time

Drivers wait in zones with thin demand



Empty repositioning

Unpaid miles moving between zones



Supply–demand imbalance

Pickups and drop-offs do not net out by zone

Core Question

How can taxi companies improve revenue efficiency and vehicle positioning by analyzing the locations, time periods, and trip patterns that generate the strongest demand and revenue?

Stakeholders: operations managers, revenue analysts, drivers

The Dataset

73.0M

Raw trip records

72.3M

Retained after cleaning

99%

Data retention rate

6

Year / month partitions

NYC Yellow Taxi Trip Records — Q1 2015 & Q1 2016

Source: NYC Taxi & Limousine Commission (TLC), collected via TPEP-authorized technology providers

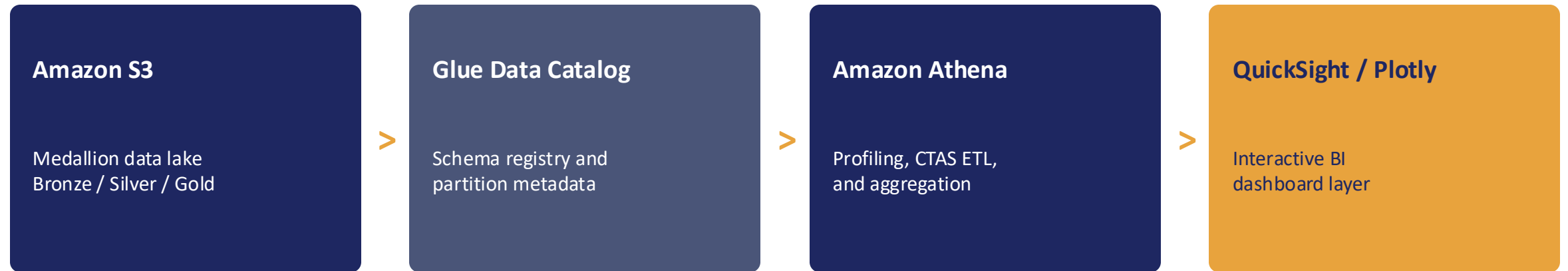
Type: Structured, transactional — one row per completed trip, ~19 columns

Reference data: NYC Taxi Zone Lookup, joined to translate location IDs into zone and borough names

Key fields: pickup/drop-off timestamps, trip_distance, location IDs, fare_amount, tip_amount, total_amount

Two matching quarters, one year apart, enable direct year-over-year comparison.

Cloud Architecture: Serverless Lakehouse



Medallion layers in S3

Bronze	Raw Parquet trip files + zone lookup, partitioned by year / month
Silver	Cleaned trip-level table (72.3M records) with calculated efficiency metrics
Gold	Pre-aggregated tables: monthly, zone, and route performance

Two Decisions That Changed the Design

01

Dropped Redshift entirely

Proposed: load a star schema into Redshift Serverless.

Built: Athena-only lakehouse. All cleaning, feature engineering, and dimensional aggregation ran as CTAS queries directly over S3 Parquet.

Why: removed cluster provisioning overhead and idle cost while still hitting sub-second query performance on Gold tables.

02

Ingested Parquet directly

Proposed: land raw CSV, then convert to Parquet with a Glue PySpark job.

Built: Parquet landed straight into Bronze, skipping the conversion job.

Why: cut landing storage size and initial scan cost immediately — the conversion step was solving a problem we could avoid creating.

Implementation Evidence

The pipeline is built and running end to end.

- 1 Ingestion** Parquet files landed in S3 Bronze, partitioned by year and month
- 2 Cataloging** Athena queried the cataloged yellow_taxi table across all six partitions
- 3 Profiling** Raw data-quality scan quantified negative fares, zero distances, invalid records
- 4 Validation** Silver confirmed clean — records violating distance, fare, and duration rules removed
- 5 Analysis** gold_zone_performance returned ranked top pickup zones

Insert your strongest screenshots here — S3 partitions, Athena partition query, quality scan, Silver validation, Gold top-zones result.

Data Quality: Validated, Not Assumed

Explicit rules, applied in Silver, with every row impact documented.

Category	Rule Condition	Impact
Duration	<code>trip_duration BETWEEN 1 AND 720</code>	Drops trips under 1 min or over 12 hrs
Spatial	<code>pickup_latitude BETWEEN 40.4 AND 40.9</code>	Excludes GPS drift outside NYC bounds
Financial	<code>fare_amount > 0 AND total_amount > 0</code>	Removes negative or fraudulent records
Joins	<code>COALESCE(zone_name, 'Unknown')</code>	Handles unmapped IDs without dropping data

72,265,156 of 73,010,683 records retained — a 99% retention rate.

Cleaning removed genuine anomalies without discarding meaningful volume, so the analysis rests on nearly the full population rather than a filtered subset.

Analytics Output

Three Gold tables drive every dashboard view.

gold_monthly_performance

Trend and year-over-year

Total trips, revenue, average distance and duration by month — the basis for 2015 vs 2016 comparison.

gold_zone_performance

Where to position

Pickup and drop-off activity by zone, plus net flow (pickups minus drop-offs) to expose supply-demand imbalance.

gold_route_performance

Which trips pay

Revenue per mile and revenue per minute by origin-destination pair, ranking genuinely efficient routes.

Metrics: total trips · total revenue · avg distance · avg duration · revenue per mile · revenue per minute · net zone flow

Key Findings

TBD

Highest-demand zones

Which pickup zones concentrate the most trips, and whether that matches where revenue efficiency is actually highest.

Peak time windows

The hours and days where revenue per minute peaks — when being on the road pays most.

Net flow imbalance

Zones that absorb drop-offs but generate few pickups, where drivers are most likely to sit idle or reposition empty.

2015 vs 2016 shift

How demand distribution and revenue efficiency moved year over year across the same quarter.

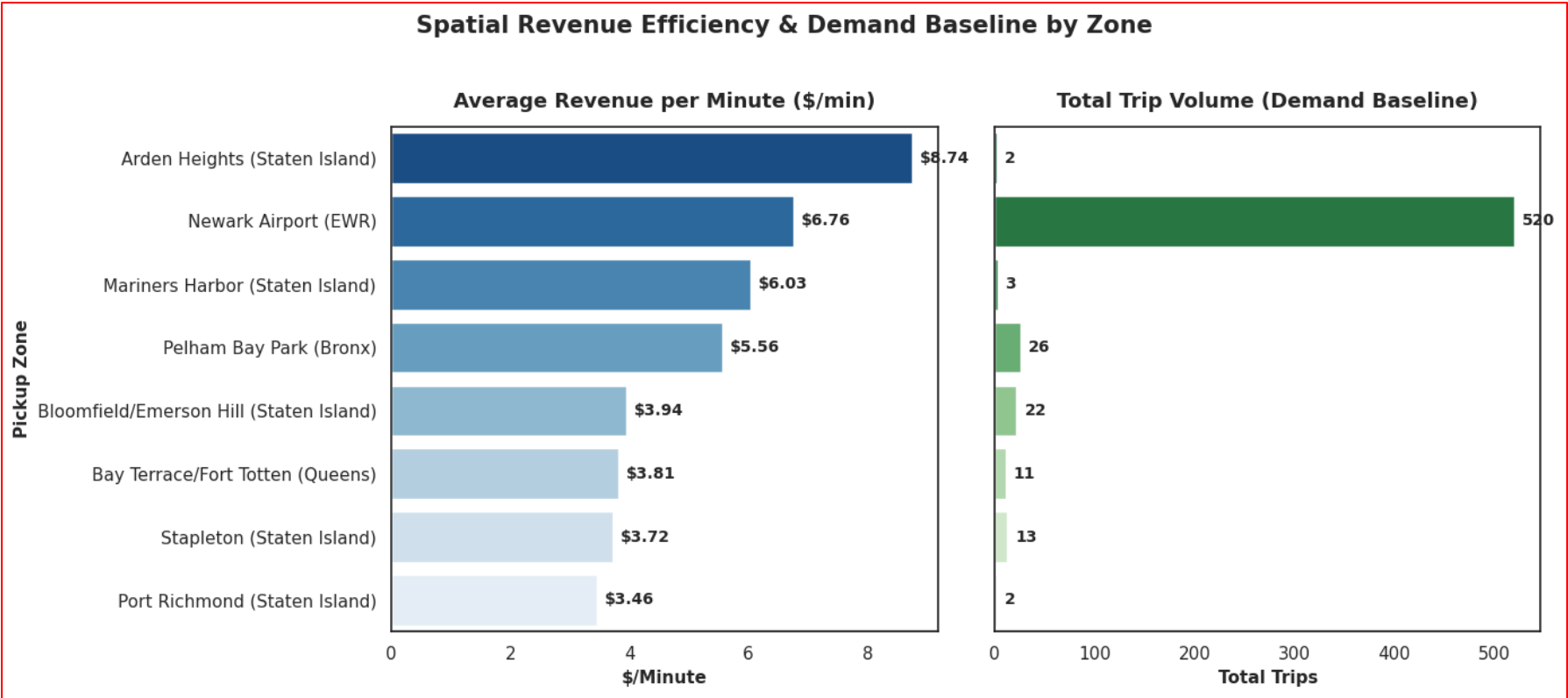
Key Finding #1 – Spatial Temporal Revenue Efficiency

"Which specific taxi zones and corridors yield the highest revenue efficiency, and how do these high-value zones shift across different times of day and days of the week?"



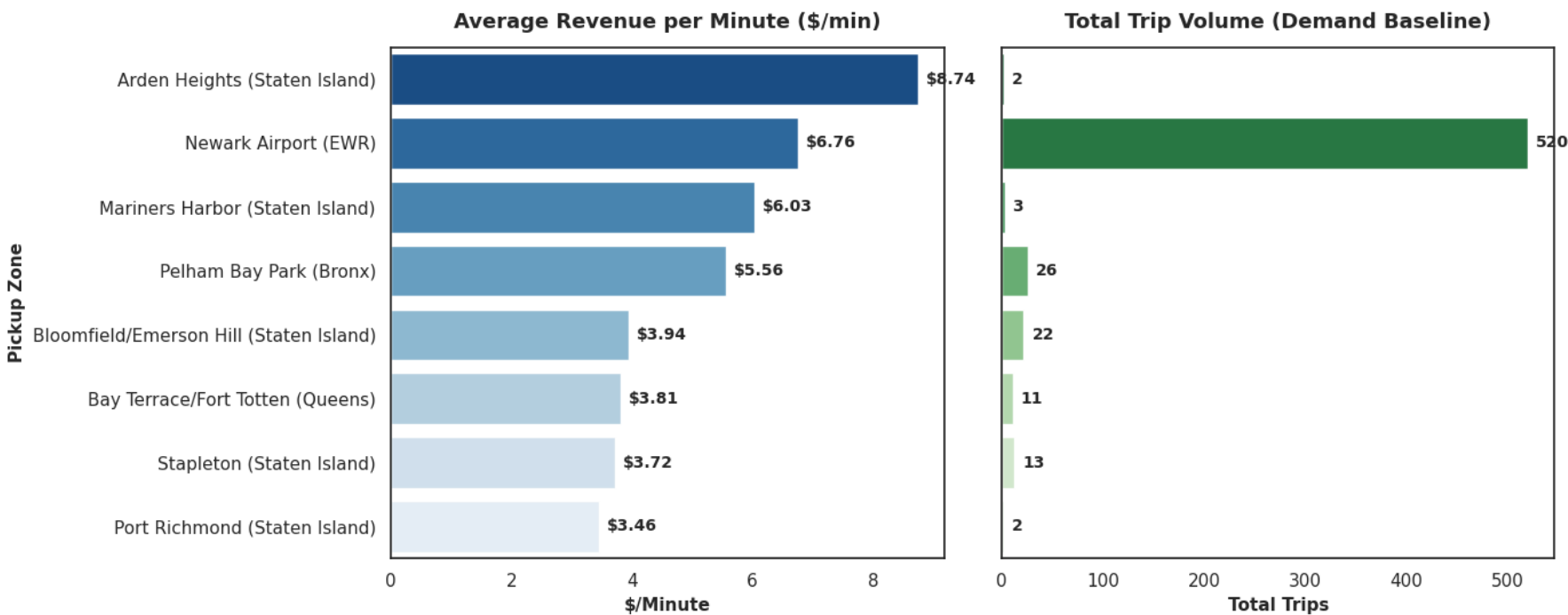
Why this matters:

- Gross revenue can be misleading if a long trip locks a driver in heavy traffic for low pay
- Analyzing revenue per minute ($\text{total_amount} / \text{trip_duration}$) and revenue per mile ($\text{total_amount} / \text{trip_distance}$) across geographic zones
- Allows fleet operators to direct drivers toward corridors that maximize hourly earnings



Key Finding #1 – Nuances & Interpretation

Spatial Revenue Efficiency & Demand Baseline by Zone



Visualization Approach

- A Dual-Subplot Horizontal Bar Chart
- Compares Revenue per minute side by side with total trip volume
 - Looking solely at unit efficiency metrics like revenue per minute can create false positives
 - Placing efficiency and trip volume side by side immediately separates statistical noise from actionable operational targets

Low-Sample Outliers:

- Low Sample Outliers: A zone like Arden Heights shows a high revenue rate (\$8.74/min), but it represents only 2 total trips in the sample

True High Yield Hubs:

- A zone like Newark Airport (EWR) maintains a strong rate (6.76/min) across 520 total trips, making it a reliable high-yield target for fleet positioning

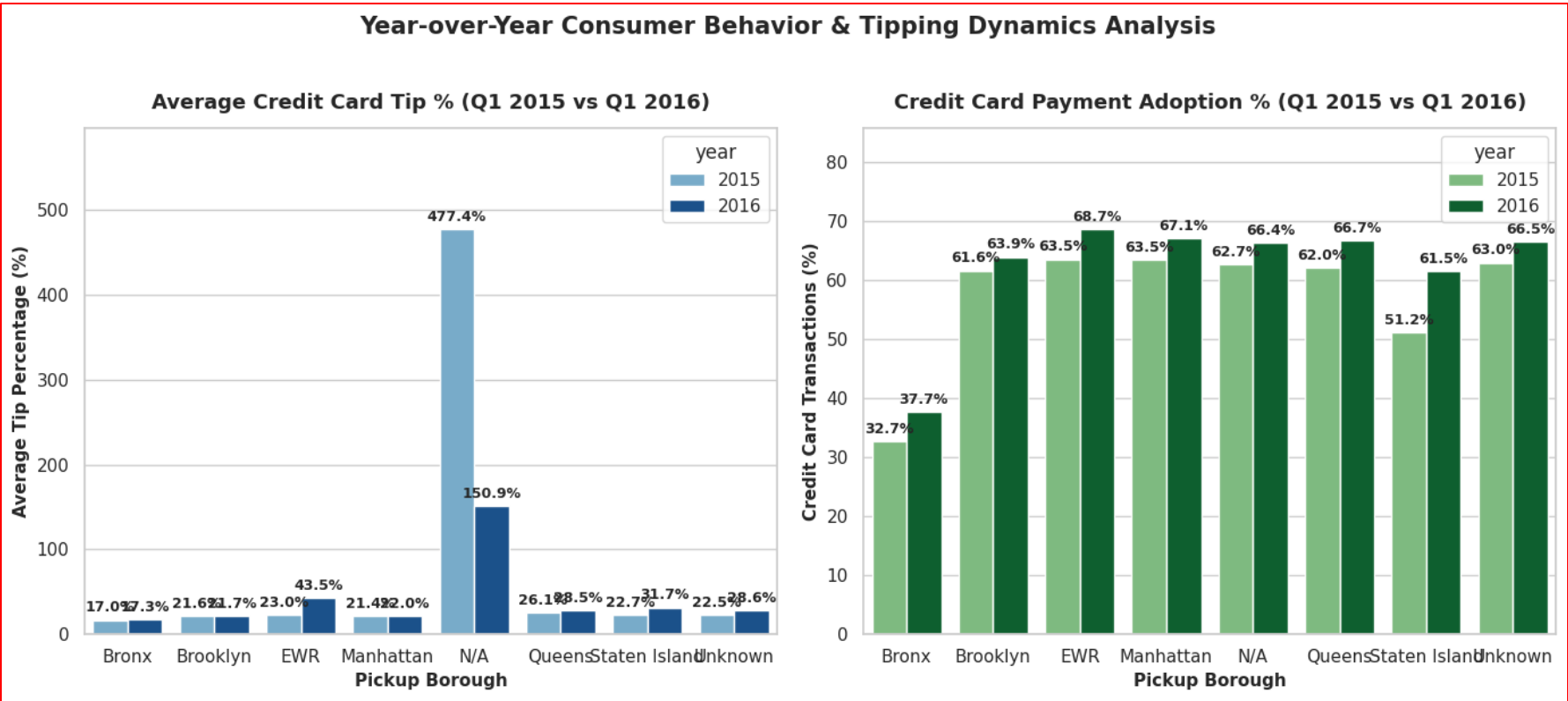
Key Finding #2 – Year-over-Year Consumer Behavior

"How did passenger tipping behavior and payment preference shift between Q1 2015 and Q1 2016 across different boroughs, and how did tip percentages impact overall driver compensation?"

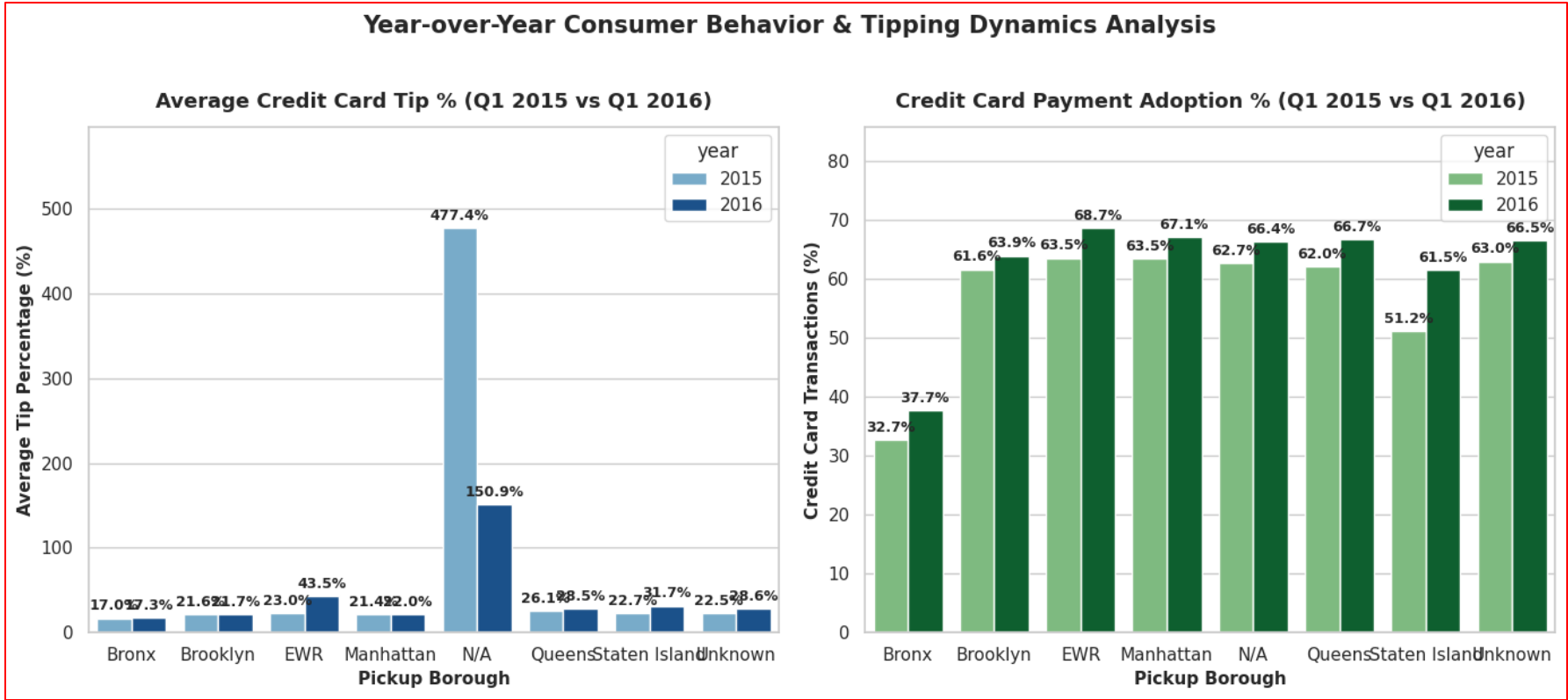


Why this matters:

- Comparing Q1 2015 against Q1 2016 datasets lets us analyze macro trends in payment method adoption
- Because cash tip amounts are typically unrecorded, filtering by credit card transactions reveals true tip percentages to evaluate total earning health
- Fields Used: payment_type, fare_amount, tip_amount, total_amount and partitioned by year and month



Key Finding #2 – Nuances & Interpretation



Visualization Approach

A Dual Grouped Bar Chart

- Allows us to analyze two core consumer shifts side by side across NYC boroughs
- Credit card adoption rose uniformly from 2015 to 2016

"N/A" Root Cause Analysis

- In N/A zones, there are likely outlier records for fare_amount
- Lookup join failure when performing a SQL left join against taxi_zone_lookup using location IDs (points outside NYC boundaries) fail to match and produce NULL
- Would recommend excluding unmapped spatial records from borough-level behavioral summaries if provided additional time
- Would recommend computing the total aggregate tip divided by total aggregate fare across each borough to prevent single low fare outliers

Average Tip Percentage:

- Isolates credit card tips relative to fare amount to uncover shifts in passenger generosity

Credit Card Adoption Rate

- Tracks macro payment trends showing the shift from cash to cashless transactions over time

Major Trade-Offs

Every architecture choice cost us something. These are the four that mattered.

Athena over Redshift

Gained zero idle cost, no provisioning, and sub-second Gold queries.

Gave up a persistent warehouse layer and the concurrency headroom a cluster gives many simultaneous BI users.

Small-file overhead

Unmanaged CTAS writes hundreds of tiny Parquet files, adding metadata overhead that slows queries and raises cost.

Mitigated by targeting 128-512 MB per Parquet object through compaction.

Partition drift

New S3 data stays invisible to Athena until the catalog updates, so queries return incomplete results with no error at all.

Mitigated with `MSCK REPAIR TABLE` after every write, or partition projection.

BI query cost

Dashboards pointed at raw or Silver data would scan enormous volumes on every refresh.

Mitigated by connecting QuickSight strictly to pre-aggregated Gold tables.

Cost Control & Cleanup

Under \$1.00

Total estimated project run cost on AWS — driven by Athena bytes scanned, not storage.

Active controls

- Partition filters enforced on every exploratory query
- AWS Budget alert set at the \$5 threshold
- BI layer restricted to Gold tables only
- Least-privilege IAM scoped to project buckets

Decommissioning protocol

1

Empty and delete
S3 buckets

2

Drop Glue tables,
database, crawlers

3

Clear Athena
result files

4

Delete custom
IAM roles

5

Verify \$0.00 in
Cost Explorer

What to Take Away

● For the business

Revenue efficiency and net zone flow tell drivers where and when to position vehicles — insight gross revenue alone cannot provide.

● For the architecture

A serverless Athena lakehouse handled 73M records end to end for under a dollar, with no cluster ever provisioned.

● For the practice

The strongest design decisions came from executing and profiling the pipeline, not from planning it on paper.